



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette:	Tarsoly Levente
Neptun-kód:	E1IK75
Szakirány:	Információs rendszerek
E-mail cím:	tarsolyl@edu.bme.hu
Konzulens(ek):	Dr. Orosz Péter, Dr. Skopkó Tamás
E-mail címe(ik):	orosz@tmit.bme.hu skopko@tmit.bme.hu

Téma címe: DDoS támadások azonosítása gépi tanulással

Feladat

Az önálló munka során a hallgató egy valós adatközpontból származó DDoS adatbázis felhasználásával kísérletezhet és megvizsgálhatja különböző gépi tanuló algoritmusok hatékonyságát. Első az adatfeldolgozás és -előkészítés fázisa, ahol az adatok tisztítása, új jellemzők létrehozása, a szükséges transzformációk elvégzése történik. A következő lépés a megfelelő gépi tanulási algoritmusok kiválasztása, implementálása és tanítása a tanító adatokon, majd a hiperparaméterek hangolása. Végül a modellek teljesítményének objektív értékelése a teszt adathalmazon, melynek eredményeit elemezve következtetéseket vonhatunk le a gépi tanulás alkalmazhatóságáról a DDoS támadások azonosításában, és javaslatokat fogalmazhatunk meg a további fejlesztésekre, esetleg valós idejű implementálhatóságra.

2024/2025. 2. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

Néhány éve a globális világhátrány miatti lezárások világszerte felgyorsították a vállalatok és intézmények digitális átállását, ennek megfelelően ezek a szolgáltatások lettek a legújabb generációs DDoS fenyegetések kiemelt célpontjai. Míg a jelenlegi DDoS támadási profilok egyes jellemzői a világhátrány időszaka előtt megjelentek, az elmúlt időszakban érték el jelenlegi összetettségüket. Az újszerű módszerek és eszközök alkalmazása mellett a támadások gyakorisága, mértéke és változékonysága is jelentősen megnövekedett.

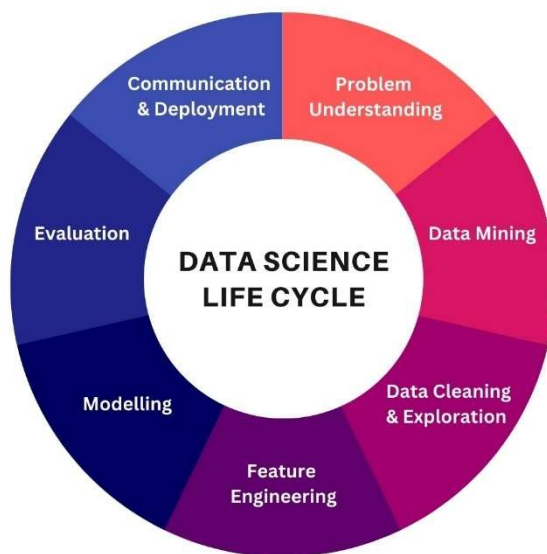
Az elosztott szolgáltatásmegtagadással járó támadások (Distributed Denial-of-Service, DDoS) célja egy online szolgáltatás (például weboldal, szerver) elérhetetlenné tétele azáltal, hogy azt túlterheli magas hálózati forgalommal [1]. Ennek a masszív forgalomnak a célja, hogy kimerítse a célpont erőforrásait, így az képtelenné válik a legitim felhasználók kéréseinek kiszolgálására. Ez azért probléma, mert a felhasználók ekkor csak azt látják, hogy a szolgáltatás, amit igénybe vennének nem működik, idővel elégedetlenné válnak és a célpont konkurenciájának szolgáltatását kezdik el igénybe venni.

A világhátrány időszakában a volumetrikus [2], szőnyegbombázó (carpet bombing) [3] és a multivektoros [2] támadások egyre népszerűbb módszerekké váltak a támadók körében, amelyek leküzdése jelentős kihívást jelent a hagyományos védelmi rendszerek számára. Ezen nehézségek kiküszöbölésében ígéretes megoldást jelenthetnek a gépi tanulási módszerek, mivel képesek a támadások komplex mintázatait generalizálni, és ezáltal detektálni olyan, korábban még nem látott, összetett támadásokat is, mint például a multivektoros incidensek.

1.2 Elméleti összefoglaló

Míg a jelenlegi DDoS detektáló eszközök és módszerek determinált szabályokra támaszkodnak a támadások felismeréséhez, a gépi tanulási algoritmusok képesek hálózati forgalmi adatokból megtanulni a támadások jellemzőit. Ehhez nagy mennyiségű, korábban elemzett (címkézett) adatra van szükség, amely tartalmazza a normális és a támadó forgalom példáit is. A megfelelően képzett gépi tanulási modellek képesek lehetnek olyan komplex mintázatokat is azonosítani, amelyek emberi szemmel nehezen észrevehetőek.

Az adatelemzés általános lépései:



Datamapu

1. ábra. Adatelemzési folyamat lépéseinek ciklikus folyamata

Az adatelemzés általában egy ciklust követ, amelyet az 1. ÁBRA szemléltet. Ez a ciklus iteratív jellegű, ami azt jelenti, hogy a tapasztalatok és az eredmények függvényében visszacsatolások történhetnek a korábbi fázisokhoz a modell finomítása érdekében.

Az első lépés a *Probléma megértése (Problem Understanding)*, amelynek során a probléma pontosan definiálásra kerül: a normális hálózati forgalom és a különböző típusú DDoS támadások közötti különbségek azonosítása.

A második lépés az *Adatbányászat (Data Mining)*. Ebben fázisban történik az adatok összegyűjtése a megfelelő forrásokból, ami ebben az esetben már előre adott.

A következő lépés az *Adattisztítás és feltárás (Data Cleaning & Exploration)*, ebben a szakaszban a meglévő adatok vizualizálása és értelmezése után történik az adathalmaz módosítása olyan módon, hogy a problémától irreleváns vagy információt nem tartalmazó adatok törlésre kerülnek, illetve az adatok olyan formátumba hozzuk őket, amely alkalmas a gépi tanulási modellek betanítására.

A negyedik lépés a *Jellemzők kinyerése (Feature Engineering)*, amikor kiemeljük azokat a jellemzőket¹, amelyek szignifikáns korrelációt mutatnak a címkével². Szintén ide tartozik új jellemzők létrehozása statisztikai módszerekkel, vagy a meglévő jellemzők közötti kapcsolatok elemzésével és felhasználásával. Ennek a lépésnek a célja, hogy a gépi tanuló modell minél jobban képes legyen mintázatokot találni, és ez alapján minél magasabb pontosságot elérni a detektálás során.

A következő fázis *Modellezés (Modelling)*. Az előkészített adatok alapján kiválasztjuk a megfelelő gépi tanulási algoritmust vagy algoritmusokat a probléma feldolgozására. A modell betanítása történik a tanító adathalmazon, és a teljesítményének mérése és hiperparamétereinek optimalizálása validációs adatok segítségével.

Az utolsó előtti fázis a *Kiértékelés (Evaluation)*. A betanított modell teljesítményét különböző metrikák alapján értékeljük a teszt adathalmazon és megvizsgáljuk, hogy a modell hogyan teljesít, valamint, hogy mennyire felel meg a kitűzött céloknak.

A legutolsó lépés a *Kommunikáció és telepítés (Communication & Deployment)*. Ennek a lépésnek a lényege, hogy az érdekelt felekkel közöljük az eredményeinket, illetve megfelelő teljesítmény esetén élesítjük a modellt a probléma megoldására.

Abban az esetben, ha nem megfelelő a folyamat végére a modellünk teljesítménye, akkor visszatérünk az első lépéshez, és az előző iterációból tanulva más stratégiát követve ismételjük meg a folyamatot.

1.3 A munka állapota, készültségi foka a félév elején

A félév kezdetén egy előre elkészített hálózati forgalmi adathalmazt kaptam. Ezt az adathalmazt egy hálózati megfigyelő eszköz rögzítette, amely determinisztikus algoritmusok segítségével már ellátta a forgalmi adatokat a megfelelő címkékkel, azaz megjelölte azokat normál forgalom, gyanús forgalom vagy DDoS támadásként.

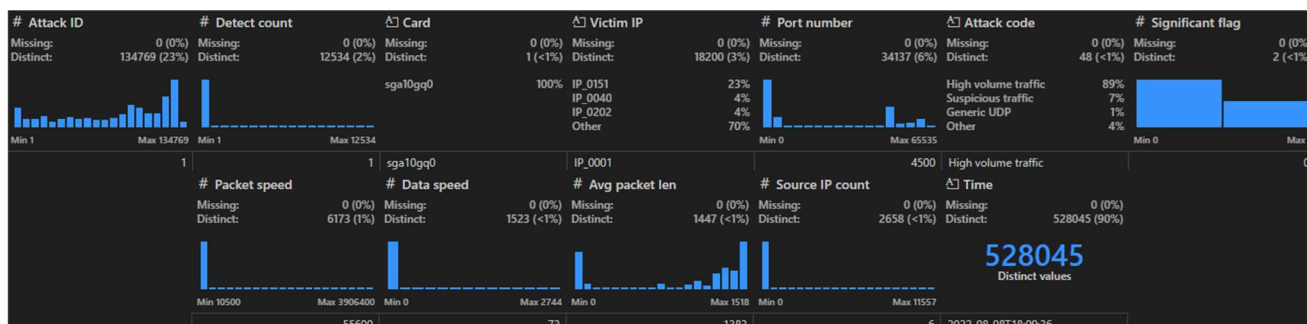
2. Az elvégzett munka és eredmények ismertetése

2.1 Adathalmaz értelmezése, vizualizáció

A kézhez kapott adathalmaz 8 fájlból állt, ez tartalmazott négy komponens (component) és négy esemény (event) fájlt. Az esemény fájlok egy hálózati eseményt tartalmaztak, a komponensek pedig egy eseménynek a részeseményeit, ha volt olyan nagy esemény, hogy azt felbontotta az eszköz, de minden eseményhez tartozott legalább egy komponens.

¹ adatelem attribútuma, tulajdonsága

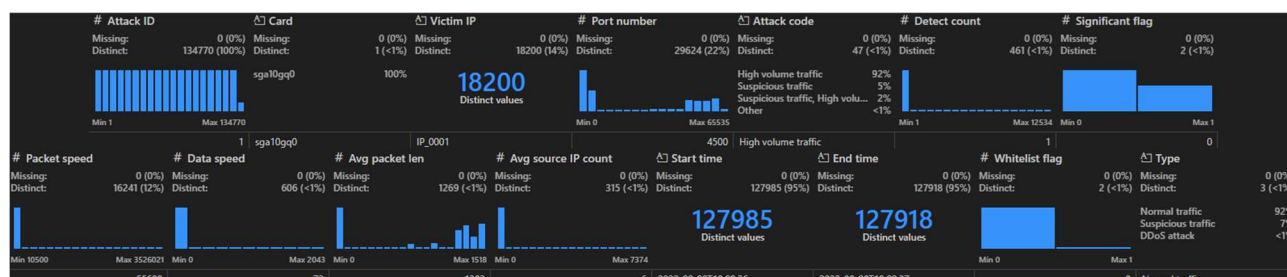
² helyes, vagy kívánt kimenet, DDoS esetében 'normál adatforgalom' vagy 'DDoS támadás'



2. ábra. A komponensek adathalmazának jellemzői az első adathalmazon.

A komponens adathalmaz jellemzőit és azokról minimális információt mutat a 2. ÁBRA:

- Attack ID: egyedi esemény azonosító. Ezzel lehet illeszteni az események táblával.
- Detect count: a komponenshez tartozó esemény komponenseinek száma
- Card: hálózati kártya azonosító. Ez minden adatsornál konstans, ezért törlésre került.
- Victim IP: anonimizált cél IP. Ezt a jellemzőt szám formátumúvá alakítottam.
- Port number: cél port száma
- Attack code: az összetevő jellege
- Significant flag: a DDoS detektor belső használatú jellemzője, erről azt az információt kaptam, hogy számomra irreleváns adat, ezért töröltem.
- Packet speed: csomagrata (csomag/másodperc)
- Data speed: adatrata (bit/másodperc)
- Avgpacket len: átlagos csomaghossz (bájt)
- Source IP count: forrás IP-k száma



3. ábra. Az események adathalmazának jellemzői az első adathalmazon.

- Time: az esemény összetevőjének kezdő ideje (dátummal)

Az események adatait és szerkezetét mutatja a 3. ÁBRA. Ahogy látható a jellemzők nagy része megegyezik a komponensekével, kivéve néhányat:

- Avg source IP count: a komponensekben érintett IP-k átlagos száma
- Start time: az esemény kezdetének ideje (dátummal). Mivel ez az oszlop a komponensekben 'Time' néven ugyanezt az információt írja le, ezért elvettem.
- End time: az esemény végének ideje (dátummal)
- Whitelist flag: a DDoS detektor belső használatú flagje, hasonlóan a 'Significant flag'-hez ezt is eltávolítottam az adathalmazból.
- Type: az esemény kategóriája (aminek a DDoS detektor felcímkézte), ahogy látható ez három értéket vehet fel:
 - Normál forgalom (Normal traffic)
 - Gyanús forgalom (Suspicious traffic)
 - DDoS támadás (DDoS attack)

2.2 Adatelőkészítés

Mivel négy adathalmazt kaptam, ezért oktatói javaslatra az első kettőt (A és B adathalmaz) használtam a modell betanítására, a harmadikat tanítás közbeni kiértékelésre (C adathalmaz), az utolsót pedig csak a félév végén végső kiértékelésre, amiből az is kiderül, hogy mennyire generalizál ténylegesen jól a modell a tanulás során (D adathalmaz).

Mikor nekikezdtem a munkának, a már korábban leírt változókat elvettem, ez után a komponenseket és eseményeket az 'Attack ID' alapján illesztettem egymással. Ezt azért gondoltam hasznos lépésnek, mert úgy véltem az esemény halmazban önmagában nincs elég jellemző, és azt reméltem akár tudom hasznosítani a komponensekben levő változókat. A másik indok, ami miatt egybeolvasztottam a két típusú táblát, mivel így nagyobb adathalmazra tettem szert, ami előnyös lehet a modell betanítása során, ha minél több adatra tud támaszkodni.

Ezt követően megkezdtem az adatok megfelelő formátumra hozását, először a szöveges változókat számmá alakítottam LabelEncoder osztály segítségével. Ennek az a feladata, hogy egy jellemzőn belül minden megjelenő szöveghez hozzárendel egy egész számot nullától növekedően, és ezt behelyettesíti a szöveg helyére (például a címkék esetében a 'DDoS attack' helyett nulla, 'Normal traffic' helyett egy és a 'Suspicious traffic' helyett kettőt helyettesített be). Ez azért fontos, mert a gépi tanuló modellek önmagukban csak számértékeket és logikai értékeket képesek értelmezni. Egyetlen másik szöveges változóm volt, az 'Attack Code'. Ennél a változónál azt vettem észre, hogy sok olyan értéke van, amely önmagában képes megadni, hogy támadás-e a vizsgált hálózati esemény, ezért ezt one-hot encoding segítségével alakítottam át. Ezzel minden egyes egyedi kategóriához létrehoz egy új bináris oszlopot. Egy adott adatpont esetén, ha az a kategória jelen van, akkor a hozzá tartozó oszlopban egyes érték szerepel, míg azokban a kategóriákban, amelyek nem tartoznak egy eseményhez, azoknál nulla.

Mivel az illesztés során sok ugyanolyan jellemzőre tettem szert, ami a komponensek és az események között is szerepelt, így soronként összehasonlítottam minden duplikált változót, és számoltam egy százalékos hasonlóságot a két oszlop között a teljes adathalmazban. Amely jellemzők egyezése nyolcvanöt százalék feletti értéket adott, annak a komponensekből tartozó változatát eltávolítottam.

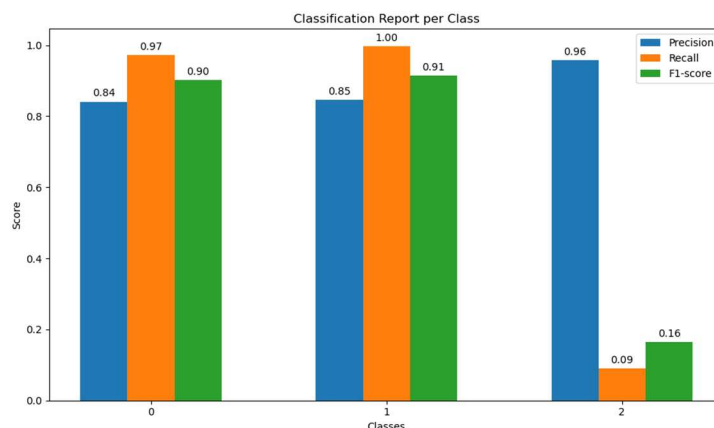
A következő lépés az üres és hibás értékek szűrése volt. Ebben az adathalmazban összesen három ilyen esemény volt, ahol dátum helyett nulla volt, ezeket a példányokat elvettem.

Az adattisztítás másik fontos lépése az anomáliák detektálása és javítása. Az anomáliák olyan értékek, amelyek az átlagoshoz képest nagyon eltérnek, ezek megnehezítik a gépi tanuló módszerek általánosítását. Ezek keresésekor arra a következtetésre jutottam, hogy az adathalmaz nem tartalmaz olyan kiugró értékeket, amelyeknek indokolt lett volna a tisztítása.

A 'Victim IP' változóból eltávolítottam a felesleges részeket, így egy numerikus változóvá alakítottam.

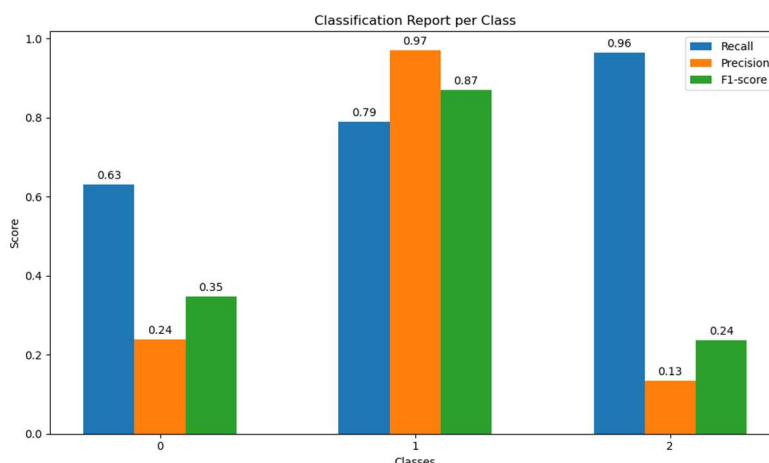
A gépi tanuló modellek számára meghatározó lépés a numerikus adatok normalizálása, mert ez garantálja, hogy minden jellemző egyenlő mértékben járul hozzá az eredményhez, megakadályozva, hogy a nagyobb nagyságrendű jellemzők elnyomják a többit. Erre a feladatra a MinMaxScaler osztály választottam, mert ez megtartja az adathalmaz eredeti eloszlását [4]. Ezen adathalmaz esetében nem minden numerikus változót normalizáltam, mert a portszám, vagy IP cím esetében nem a szám nagysága hordoz információt, hanem a konkrét értéke. Azokat a jellemzőket, amelyek nagyságrendbeli jelentést hordoznak, mint például a 'Data speed' vagy 'Packet speed', ezeket normalizáltam.

Adatelemzési folyamatnál fontos egy baseline érték megállapítása, ehhez egy fa alapú modellt választottam, ami végül a Véletlen erdő (Random Forest) osztályozó lett. Ez a módszer sok döntési fára tanítja az adat részhalmazait, és ezek a fák szavazzák meg a végső eredményt. Ez azért is működik jól, mert nem okoz neki problémát a kiegyensúlyozatlan adat [5].



4. ábra. Random forest pontossága, találati aránya és F1-értéke osztályonként az adatelőkészítés után.

A 4. ÁBRA mutatja, meglepően jó eredmények születtek, ami az 'Attack Code' változónak köszönhető, mert ez olyan jellemző, amely képes megadni, hogy egy DDoS támadás-e.

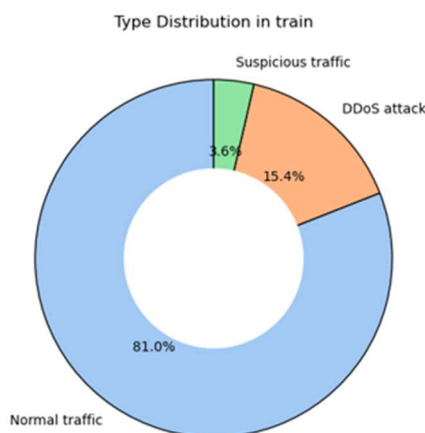


5. ábra. Random forest pontossága, találati aránya és F1-értéke osztályonként az 'Attack Code' eltávolítása után.

Az 5. ÁBRA jellemzi, mennyit változott az eredmény az 'Attack Code' eliminálásakor a DDoS támadások detektálásán, a normál forgalmat még így is jól megtalálja az algoritmus.

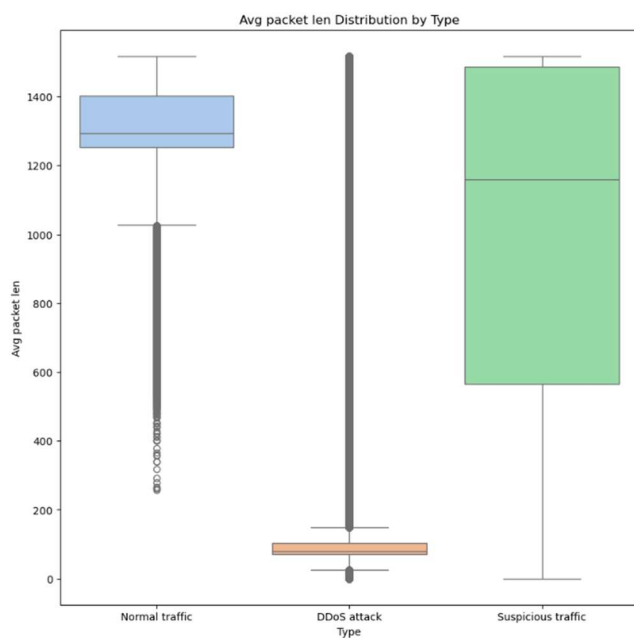
2.2 Vizualizáció

Az adatok előkészítése és tisztítása után fontos lépés azok értelmezése, korrelációk keresése, hogy utána megfelelő jellemzőket „erősítsünk meg” a feature engineering során.

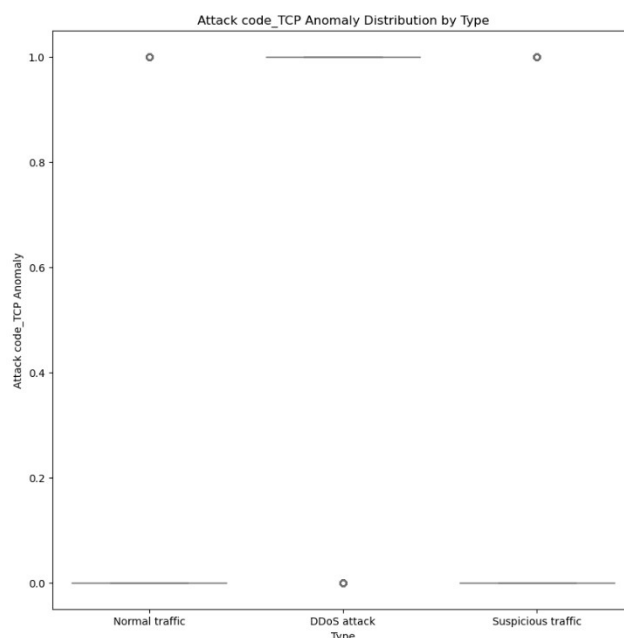


6. ábra. A tanító adatokban a forgalomtípusok aránya.

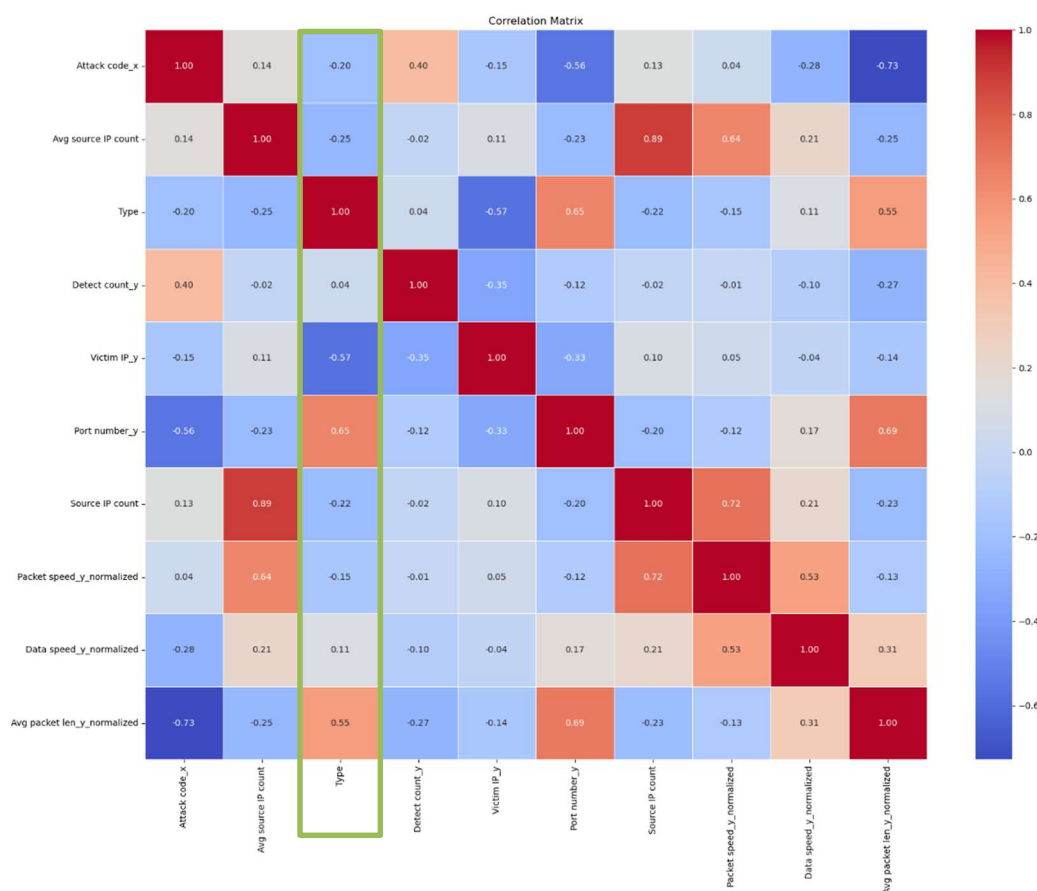
A 6. ÁBRA mutatja, hogy elég kiegyensúlyozatlan az osztályok aránya, ennek következtében könnyen elképzelhető, hogy adatdúsítással javítható az eredmény. Az alul mintavételezésnek van egy olyan nehézsége, hogy a hálózati forgalom nagyon szerteágazó, emiatt nehéz kiszűrni úgy a törlendő adatokat, hogy ne töröljünk ki egy konkrét típusú forgalmat.



8. ábra. 'Avg packet len' eloszlása forgalomtípusonként.



9. ábra. 'Attack code' egyik változatának eloszlása forgalomtípusonként.



7. ábra. Jellemzők egymással és a célváltozóval való korrelációja. A zölddel jelölt oszlop a címke és a jellemzők közti korrelációt mutatja be.

A 8. ÁBRA mutatja, hogy léteznek olyan jelenlegi változók, amelyek segítségével szemmel láthatóan elkülöníthetők az osztályok egy része, viszont a 9. ÁBRA alátámasztja a már korábban említett problémát, hogy az 'Attack code' sok esetben egyértelműsíti, hogy egy forgalom DDoS vagy sem, ezért töröltem az 'Attack code' jellemzőt, és annak one-hot enkódolt változatát.

A 7. ÁBRA a jellemzők korrelációit mutatja. Ennek segítségével könnyedén meg lehet állapítani azokat a jellemzőket, amelyek értéke lineáris kapcsolatban áll a célváltozóval, ennek következtében azokra érdemes nagyobb hangsúlyt fektetni például feature engineering során. Az ábrán látható, hogy a három „legfontosabb” változó az az 'Avg packet len', a 'Victim IP' és a 'Port number'.

2.3 Jellemzők kinyerése

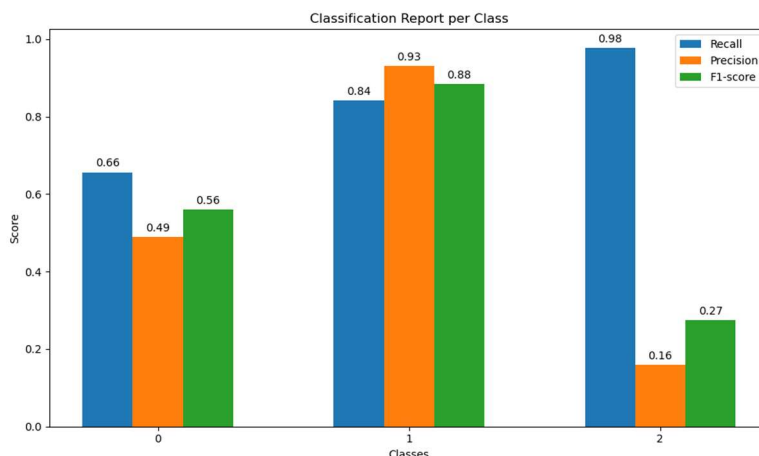
A jellemzők kinyerése során két irányba indultam el, az egyik a vizualizáció során látott 'Avg packet len' és a többi hálózati metrikával kapcsolatos jellemzőkből újak generálása, illetve az idő és időtartam kihasználása új jellemzők létrehozásához.

Az idővel kapcsolatosan először egy időtartamot hoztam létre, hogy meddig tartott egyes esemény, ez után pedig olyan változókat hoztam létre, mint a napszak, hét melyik napja és hétvége-e a vizsgált adat. Ekkor lettem arra figyelmes, hogy vannak olyan események, amikor az időtartam 0. Ez azért lehetséges, mert az adathalmaz másodpercre adja meg, hogy mikor kezdődött és végződött a hálózati esemény, viszont rengeteg olyan folyamat van, ami akár milliszekundumok alatt véget ér, sőt a volumetrikus támadások is így működnek, ezért ezekben az esetekben az időtartamot 0.1-re állítottam.

A hálózati metrikák segítségével eleinte csak statisztikai jellemzőket generáltam eseményenként (átlagos, medián, minimum, maximum). Később eszembe jutott, hogy akár az idővel is arányosíthatnám ezeket a változókat, mint például hány csomag érkezett be másodpercenként vagy csomagszám per adatsebesség.

Mivel a korrelációknál a két legnagyobb korrelációt a port szám és az IP cím mutatta, ezekkel is generáltam új jellemzőket, ezeknek a teljes adathalmazban levő arányát/ritkaságát néztem meg és adtam hozzá a jellemzők közé.

Az előbb említett módszereken kívül annak ellenére, hogy dimenziócsökkentésre használt eszköz, lehet, hogy jól kiemeli egyes jellemzők tulajdonságát, PCA [6] segítségével generáltam még öt új jellemzőt.



10. ábra. Random forest pontossága, találati aránya és F1-értéke osztályonként a jellemzők kinyerése után.

Mint az adatelőkészítés után, most is leteszteltem az adathalmazt, ennek eredményeit a 10. ÁBRA mutatja. Lényeges előrelépés észrevehető a DDoS osztály esetében, sajnos a gyanús forgalom detektálása még mindig nehezebbé esik a modellnek.

2.4 Adatdúsítás

Mivel a 6. ÁBRA által észrevehető, hogy szignifikánsan el vannak tolódva az arányok az osztályok között, ezért arra jutottam, hogy megpróbálkozom a szintetikus adatokkal való bővítéssel. Erre a célra a SMOTE [7] algoritmust választottam, mert ez tűnt a legelterjedtebbnek és legkönnyebben implementálhatónak. Próbáltam nem a többségi osztállyal megegyező számú mintára hozni a kisebbséget, mert ez túl sok szintetikus adatot generált volna. Emiatt próbáltam a nagyságrendeket megtartani, és kísérleteztem 2szeres, 1.7szeres, 1.5szeres és 1.2szeres növeléssel. Ezeket ugyanúgy RandomForest segítségével végeztem el, illetve hozzávettem a kiértékelő modellek közé egy Boosting alapú módszert is, ami a GradientBoosting modell lett.

F1 SCORE	SMOTE *1.2	SMOTE *1.5	SMOTE *1.7	SMOTE *2
RandomForest	0.5825	0.5972	0.5952	0.5917
GradientBoosting	0.5157	0.5180	0.5118	0.5062

1. táblázat. SMOTE algoritmus F1 pontszáma egyes mintaszám növelési értékeknél RandomForest és GradientBoosting modellekkel.

Az 1. TÁBLÁZAT alapján egyértelműen eldönthető, hogy a másfélszeres adatszám növelés az ideális ezen modellek esetében, ezért ezt alkalmaztam a munka további részében.

2.5 Gépi tanuló modellek implementálása

Ebben a részben nem csak a modellek implementálása és kiértékelés volt a feladat, hanem azok hiperparamétereinek optimalizálása. Erre a GridSearch algoritmust használtam, aminek egyik hátránya, hogy mivel keresztvalidációt használ, ezért nagyon magas a futásideje.

F1 SCORE	DDoS	Normal	Suspicious	Macro avg	Accuracy	
RandomForest	0.56	0.88	0.27	0.57	RFC	0.81
GradientBoosting	0.56	0.86	0.12	0.51	GBM	0.78
XGBoost	0.53	0.87	0.35	0.58	XGB	0.79
LightGBM	0.57	0.87	0.30	0.58	LGB	0.80
CatBoost	0.52	0.83	0.16	0.50	CAT	0.74
Simple NN	0.44	0.88	0.32	0.55	Simple	0.81
2 Dense réteg	0.0	0.85	0.0	0.28	2 Dense	0.73

2. táblázat. Modellek F1 pontszáma osztályonként és egyesítve.

3. táblázat. Modellek pontossága.

A 2. TÁBLÁZAT és 3. TÁBLÁZAT tartalmazza a kipróbált modellek listáját és azok teljesítményét. A már használt modelleken kívül a boosting alapú megoldásokkal is próbálkoztam. Ezek között az XGBoost és a LightGBM is megfelelően teljesített.

Ezt követően próbálkoztam mélytanulással is. Először egy olyan hálóval próbálkoztam meg, amelynek a rejtett rétegei egy 256 és egy 128 neuronból álló réteg ReLU aktivációs függvényvel. Ez először nem is futott le, mert nem volt megfelelő a bemenet formája, ezért elkezdtem egy olyan hálón keresni a megfelelő bemeneti formát, csak a bemeneti és a 3 neuronos softmax kimeneti rétege van. Ahogy a 2. TÁBLÁZAT mutatja, végezetül ez sokkal jobb eredményt ért el, mint az eredeti hálóm, bár a gépi tanuló módszerek így is pontosabbak voltak.

2.6 Második iteráció

Azt sejtettem, hogy a port számok segítségével még tudnék javítani az eredményemen. Ezért a 'Port number' változóból hálózati protokollok szerint one-hot enkódoltam a jellemzőt.

A másik változtatás a hiperparaméterek optimalizációjában történt, mivel az Optuna [8] csomagot kezdtem el használni. Ennek előnye, hogy minden paraméterre a megadott tartományon belül a korábbi próbák eredményeit felhasználva okosabban választja ki a következő kipróbálandó hiperparamétereket. Így gyorsabban találhatja meg az jó paramétereket, kevesebb kiértékeléssel. Ezzel lényegesen meg tudtam gyorsítani az optimalizációs folyamatot.

<i>F1 score</i>	<i>DDoS</i>	<i>Normal</i>	<i>Suspicious</i>	<i>Macro avg</i>
<i>RFC</i>	0.67	0.86	0.32	0.62
<i>XGB</i>	0.59	0.88	0.32	0.60
<i>LGB</i>	0.63	0.87	0.31	0.61

4. táblázat. Modellek F1 pontszáma osztályonként és egyesítve a második iteráció után.

2.7 Harmadik iteráció

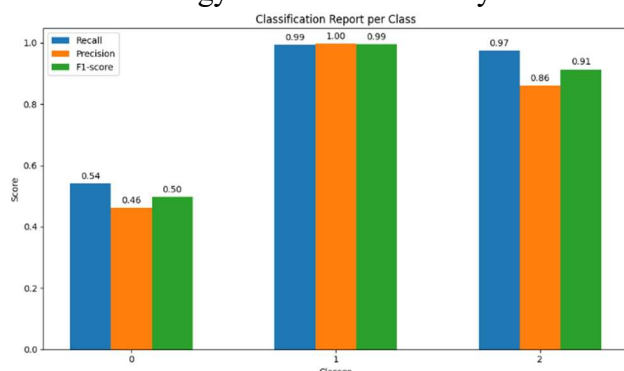
Ebben a szakaszban próbáltam kísérletezni más módszerekkel. Először mivel elvettem az 'Attack code' változót, megpróbálkoztam ennek visszahozásával. Erre az ötletem a változó becslése volt gépi tanulással. RandomForest segítségével meg is próbálkoztam vele, de az egy százalékos F1 pontszám elérése után elvettem az ötletet. A probléma itt is valószínűleg sokféle értéken és alacsony mintaszámon alapul.

Másodszor az együttes tanulást próbáltam még ki az adathalmazon, azon belül a szavazást. Ennek eredményeit az 5. TÁBLÁZAT mutatja.

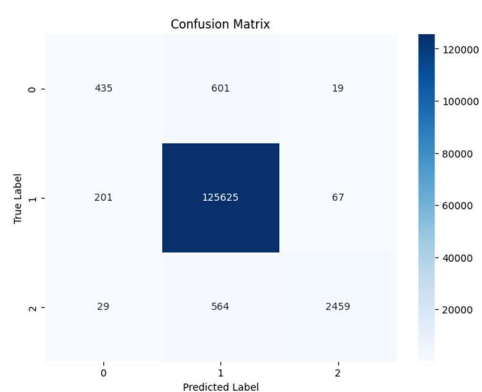
<i>F1 SCORE</i>	<i>DDoS</i>	<i>Normal</i>	<i>Suspicious</i>	<i>Macro avg</i>
<i>Voting soft</i>	0.78	0.94	0.15	0.62
<i>Voting hard</i>	0.78	0.84	0.11	0.61

5. táblázat. Együttes tanulás eredményei.

Végezetül megnéztem azt, hogy milyen eredményt hoz a RandomForest, ha nem illesztem az eseményeket és a komponenseket, csak az eseményen futtatom. Ez az adatelőkészítés fázisában egyáltalán nem hozott jó eredményt, viszont a feature engineering és augmentáció lefuttatásával egy szokatlan eredményre lettem figyelmes.



11. ábra. F1 pontszám a forgalomtípusokon csak az események adathalmazán feature engineering és adatlúsítás után.



12. ábra. Újonnan felfedezett megoldás konfúziós mátrixa.

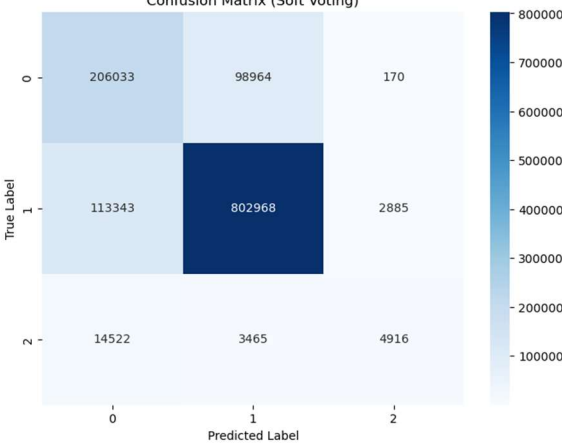
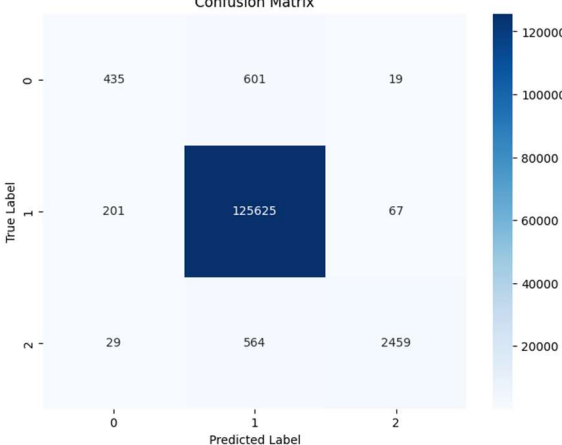
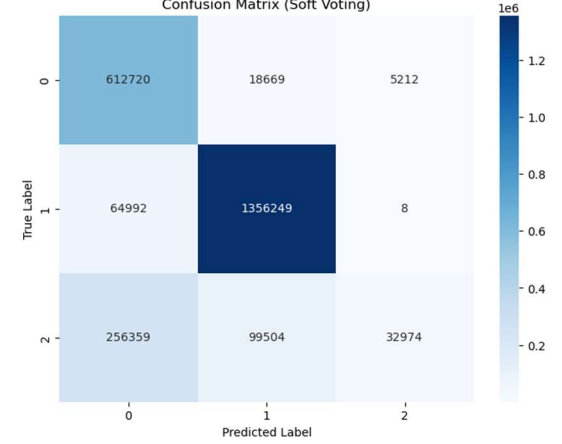
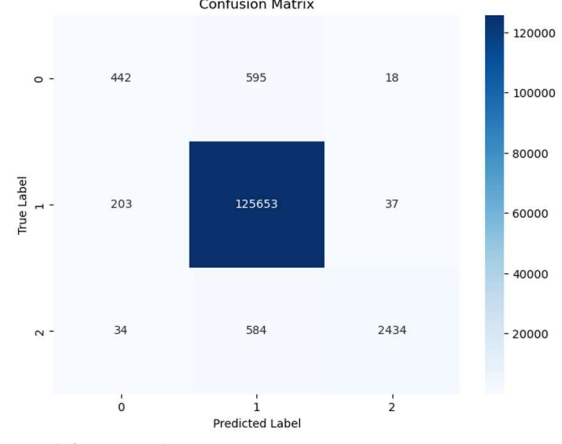
Ahogy a 11.ÁBRA is reprezentálja, teljesen más eredményeket kaptam, mint eddigi kiértékeléseim során. A normál forgalmat és a gyanús forgalmat majdnem tökéletesen képes detektálni, viszont a DDoS teljesítménye leesett. A probléma ezzel a megoldással a konfúziós mátrix vizsgálatával derült ki igazán, amit a 12. ÁBRA jelenít meg. Gyakorlatilag nagyjából minden forgalmat normál forgalomnak osztályoz, a gyanús forgalmat még képes detektálni. Ez egyébként egy éles rendszerben jól implementálható megoldás abból a szempontból, hogy nem zárja ki a normál forgalmat, de DDoS detekcióban volt erősebb eredményem.

2.8 Összefoglalás

A feladat a DDoS támadások determinált algoritmusokkal való detektálását segítő gépi tanulási módszer megalkotása volt. Ez amiatt fontos probléma, hogy napjainkban olyan összetettek és sokszínűek a támadások, hogy ezeket egyre nehezebb előre definiált módszerekkel detektálni.

A félév elején kapott adathalmazt elemezve, tisztítva és a gépi tanulási modellek számára

megfelelő formátumra hozva kezdtem meg a tényleges munkát. A jellemzők kinyerése után lehetőségem volt megismerkedni az adatdúsítással, illetve új modellekkel. Ezt követően többször végig iteráltam az adatelemzés folyamatán, igyekeztem minél jobb eredményt elérni.

 13. ábra. A szavazással elért eredmény konfúziós mátrixa a tesztalmazon.	 14. ábra. Csak eseményeken mért eredmény RandomForest modellel konfúziós mátrixa a tesztalmazon.
Accuracy: 0.8129 Precision: 0.7069 Recall: 0.5878 F1-Score: 0.6124	Accuracy: 0.9886 Precision: 0.8703 Recall: 0.7386 F1-Score: 0.7929
 15. ábra. A szavazással elért eredmény konfúziós mátrixa a végső kiértékelésen.	 16. ábra. Az eseményeken mért eredmény RandomForest modellen konfúziós mátrixa a végső kiértékelésen.
Accuracy: 0.8182 Precision: 0.8130 Recall: 0.6672 F1-Score: 0.6237	Accuracy: 0.9887 Precision: 0.8732 Recall: 0.7382 F1-Score: 0.5548

6. táblázat. A legjobb végső eredmények a teszt adathalmazon.

Úgy gondolom, még van lehetőség a fejlődésre, akár a két végső módszer egyesítésével, bináris osztályozással, illetve az idősor egy teljesen új perspektívát tudna adni a feladatnak.

Összességében a végső eredményeket a 6. TÁBLÁZAT mutatja, ezek a probléma megközelítésétől függően voltak jók. Az eredeti megoldásommal inkább a DDoS detekció, és generalizálhatóság a lényeg, a második esetben egy valós környezetben jól implementálható megoldást adtam meg, mert kis arányban detektálja a normál forgalmat DDoSként.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

A tanulmányozott irodalom jegyzéke:

- 1] Cloudflare, „Learning: DDoS,” Cloudflare, [Online]. Available: <https://www.cloudflare.com/en-gb/learning/ddos/what-is-a-ddos-attack/>. [Hozzáférés dátuma: Február 2025].
- 2] T. Emmons, „2021: Volumetric DDoS Attacks Rising Fast,” Akamai Technologies, 31 Március 2021. [Online]. Available: <https://www.akamai.com/blog/security/2021-volumetric-ddos-attacks-rising-fast>. [Hozzáférés dátuma: Február 2025].
- 3] Juniper Networks, Corero Network Security PLC, „2023 DDoS Threat Intelligence Report,” Juniper Networks, 2024. [Online]. Available: <https://www.juniper.net/content/dam/www/assets/analyst-reports/us/en/2023/corero-ddos-threat-intelligence-report.pdf>. [Hozzáférés dátuma: Május 2025].
- 4] GeeksforGeeks, „Data Normalization Machine Learning,” GeeksforGeeks, 4 November 2024. [Online]. Available: <https://www.geeksforgeeks.org/what-is-data-normalization/>. [Hozzáférés dátuma: Február 2025].
- 5] R. Feki, „Imbalanced data: best practices,” Medium, 3 Január 2022. [Online]. Available: <https://rihab-feki.medium.com/imbalanced-data-best-practices-f3b6d0999f38>. [Hozzáférés dátuma: Február 2025].
- 6] GeeksforGeeks, „Principal Component Analysis(PCA),” GeeksforGeeks, 3 Február 2025. [Online]. Available: <https://www.geeksforgeeks.org/principal-component-analysis-pca/>. [Hozzáférés dátuma: Május 2025].
- 7] J. C. Olamendy, „How to Tackle Unbalanced Data with SMOTE: A Comprehensive Guide,” Medium, 5 December 2023. [Online]. Available: <https://medium.com/@juanc.olamendy/how-to-tackle-unbalanced-data-with-smote-a-comprehensive-guide-706347ad37ad>. [Hozzáférés dátuma: Április 2025].
- 8] Juniper Networks, Corero Network Security PLC, „2019 Full Year DDoS Trends Report,” Juniper Networks, 2020. [Online]. Available: <https://www.juniper.net/content/dam/www/assets/validation-reports/us/en/2020/2019-full-year-ddos-trends-report.pdf>. [Hozzáférés dátuma: Május 2025].
- 9] K. Pykes, „Data Preprocessing: A Complete Guide with Python Examples,” DataCamp, 15 Január 2025. [Online]. Available: <https://www.datacamp.com/blog/data-preprocessing>. [Hozzáférés dátuma: Február 2025].